

Deterministic polynomial-time algorithm for exact matching in bipartite graphs

W4414427837_p0 (website page)

Website #29 | Solver verdict: SOLVED | Verifier verdict: ISSUES | Shown: solver output

Problem

Given a bipartite graph $G = (A, B; E)$ with $|A| = |B| = n$, every edge colored red or blue, and an integer k , design a deterministic polynomial-time algorithm that

- outputs a perfect matching $M \subseteq E$ with exactly k red edges, i.e. $r(M) = k$, if such an M exists; and
- otherwise correctly outputs that no perfect matching of G has red count exactly k .

Equivalently, on a No-instance the algorithm must certify

$$\forall \text{ perfect matchings } N \subseteq E, \quad r(N) \neq k.$$

Alternative readings considered

The mathematical statement is unambiguous. The only operational ambiguity is whether “design an algorithm” permits using results newer than the paper mentioned in the prompt. I adopt the strongest reading: solve the unrestricted bipartite problem itself. I do use a 2026 theorem as a cited step, and I separate that imported decision theorem from the search reduction proved here.

Related literature found

- **Du (2026)** — **SOLVES**. A recent arXiv preprint, submitted on April 2, 2026 and revised on April 9, 2026, claims a deterministic $O(n^6)$ algorithm for Exact Matching on all bipartite graphs. Its algorithmic outline reduces general bipartite graphs to brace blocks via tight-cut decomposition and then tests each block by determinant interpolation. This is the essential external decision result used below. (arxiv.org)
- **Jia–Svensson–Yuan (SODA 2023)** — **BACKGROUND**. Their abstract states that, at that time, no deterministic polynomial-time algorithm was known even for bipartite exact matching; this matches the historical status described in the prompt. (epubs.siam.org)
- **El Maalouly–Steiner (MFCS 2022)** — **PARTIAL**. They obtain deterministic polynomial time only for graphs of bounded independence number, and for bipartite graphs of bounded bipartite independence number; this does not cover arbitrary bipartite graphs. (drops.dagstuhl.de)
- **Dürr–El Maalouly–Wulf (2023)** — **PARTIAL**. They give deterministic approximation algorithms for the number of red edges in a perfect matching, not an exact algorithm for arbitrary k . (arxiv.org)

- **Papadimitriou–Yannakakis (1982)** — BACKGROUND. This is the original source introducing the exact matching problem in the course of studying restricted spanning tree problems. (cir.nii.ac.jp)
- **Mulmuley–U. Vazirani–V. Vazirani (1987)** — BACKGROUND. They gave the classical randomized polynomial-time algorithm, so this source does not meet the deterministic requirement. (dblp.org)

Proof sketch

The genuinely hard part is deterministic *decision*. Up through the older literature I checked, the status was exactly as in the prompt: no deterministic polynomial-time algorithm was known for arbitrary bipartite instances. However, a very recent preprint of Du claims such a deterministic algorithm for all bipartite graphs, via tight-cut decomposition into brace blocks and a determinant-based nonvanishing theorem. I use that result only as a black-box decision routine. So this manuscript is *RELIES-ON-EXTERNAL-PROOF* for the decision layer. (epubs.siam.org)

My independent contribution is the search reduction that turns deterministic decision into deterministic witness extraction. The key lemma is elementary: forcing an edge $e = ab$ into the perfect matching is equivalent to deleting a and b and reducing the red target by 1 exactly when e is red. Therefore, if the current residual instance is feasible and we choose any remaining left vertex a , then at least one edge incident to a must preserve feasibility under this reduction—namely the edge used by some witness perfect matching. Testing the incident edges one by one with the decision routine identifies such an edge deterministically. Repeating this for n rounds constructs a perfect matching.

This avoids tempting but false shortcut heuristics such as “the achievable red counts form an interval.” They need not: two disjoint alternating 4-cycles can realize only $\{0, 2, 4\}$. The proof below uses no such claim; it uses only the edge-forcing bijection and induction on the number of unmatched vertices.

Main result

Theorem 1. *There exists a deterministic polynomial-time algorithm that, given a bipartite graph $G = (A, B; E)$ with red/blue edge colors, $|A| = |B| = n$, and an integer k , decides whether G has a perfect matching M with $r(M) = k$, and outputs such an M when it exists.*

More precisely: if Dec is any deterministic decision algorithm for the bipartite exact matching problem, then one can output a witness using at most $m + 1$ calls to Dec, where $m = |E|$. Applying this to Du’s deterministic polynomial-time decision algorithm yields the claimed deterministic polynomial-time search algorithm.

Proof

For an edge e , let

$$\rho(e) = \begin{cases} 1, & \text{if } e \text{ is red,} \\ 0, & \text{if } e \text{ is blue.} \end{cases}$$

Lemma 1 (forcing one edge). *Let $H = (X, Y; F)$ be a bipartite graph with $|X| = |Y|$, let $t \in \mathbb{Z}$, and let $e = xy \in F$. Then the following are equivalent:*

1. H has a perfect matching N such that $e \in N$ and $r(N) = t$.
2. The graph $H - \{x, y\}$ has a perfect matching N' with

$$r(N') = t - \rho(e).$$

Proof. Assume (1). Then $N' := N \setminus \{e\}$ covers every vertex of $H - \{x, y\}$ exactly once, because N covered every vertex of H exactly once and e was the unique edge of N incident to x and to y . Hence N' is a perfect matching of $H - \{x, y\}$. Since removing e removes exactly $\rho(e)$ red edges, we get

$$r(N') = r(N) - \rho(e) = t - \rho(e).$$

Conversely, assume (2). Let $N := N' \cup \{e\}$. The matching N' covers every vertex of $H - \{x, y\}$ exactly once, and the edge e covers x and y . Since x and y are not vertices of $H - \{x, y\}$, no edge of N' is incident to either of them, so N is again a matching. It covers every vertex of H exactly once, hence it is a perfect matching of H . Its red count is

$$r(N) = r(N') + \rho(e) = t.$$

So (1) holds. □

Proof of the theorem. Fix once and for all total orders on A and B . We first describe the generic search reduction from decision to witness extraction.

Decision subroutine. Let $\text{Dec}(H, t)$ denote a deterministic algorithm that decides whether a bipartite red/blue graph H with equal side sizes has a perfect matching with exactly t red edges.

Hypothesis audit for the cited decision theorem. Du's theorem is stated for bipartite graphs with equal side sizes, red/blue edge coloring, and an integer target. Every oracle instance produced below is obtained from the original input by deleting one vertex from the left side and one from the right side at each step, while keeping inherited edge colors. Therefore each residual instance is still bipartite, still has equal side sizes, and still has an integer target. Hence Du's theorem applies at every oracle call. (arxiv.org)

Algorithm Search. On input (G, k) :

1. Run $\text{Dec}(G, k)$. If the answer is No, output No and halt.
2. Initialize $H := G$, $t := k$, and $M := \emptyset$.
3. While $X := A(H)$ is nonempty, do the following:
 - (a) Let a be the smallest vertex of $A(H)$.
 - (b) Scan the edges $e = (a, b) \in E(H)$ in increasing order of $b \in B(H)$.
 - (c) For the first edge $e = (a, b)$ such that

$$\text{Dec}(H - \{a, b\}, t - \rho(e)) = \text{Yes},$$

set

$$M := M \cup \{e\}, \quad H := H - \{a, b\}, \quad t := t - \rho(e),$$

and continue to the next iteration of the while-loop.

4. When the loop ends, output M .

I now prove correctness.

Invariant. After each completed iteration of the while-loop, the following hold:

- (I1) M is a matching in the original graph G .
- (I2) H is obtained from G by deleting exactly the endpoints of the edges in M .
- (I3) $r(M) + t = k$.
- (I4) H has a perfect matching with exactly t red edges.

Before the first iteration, step 1 has certified $\text{Dec}(G, k) = \text{Yes}$. Thus $H = G$ has a perfect matching with exactly k red edges, so (I4) holds. Also $M = \emptyset$, hence (I1), (I2), and (I3) hold.

Assume the invariant holds at the start of some iteration, and let a be the chosen smallest vertex of $A(H)$. By (I4), there exists a perfect matching N of H with $r(N) = t$. In that matching, the vertex a is incident to a unique edge $e^* = (a, b^*) \in N$. By the lemma, applied to H , t , and e^* ,

$$H - \{a, b^*\}$$

has a perfect matching with exactly $t - \rho(e^*)$ red edges. Therefore

$$\text{Dec}(H - \{a, b^*\}, t - \rho(e^*)) = \text{Yes}.$$

So the inner scan is guaranteed to find at least one acceptable edge; the algorithm cannot get stuck.

Suppose the algorithm selects some edge $e = (a, b)$ for which the oracle answered Yes. By the lemma again, $H - \{a, b\}$ has a perfect matching with exactly $t - \rho(e)$ red edges. Hence, after the update

$$H := H - \{a, b\}, \quad t := t - \rho(e),$$

property (I4) remains true.

Property (I1) remains true because the newly added edge e is disjoint from every earlier edge in M : by (I2), all earlier endpoints have already been deleted from H , while e lies in H . Property (I2) is true by construction of the update. Finally,

$$r(M \cup \{e\}) + (t - \rho(e)) = r(M) + \rho(e) + t - \rho(e) = r(M) + t = k,$$

so (I3) is preserved.

Thus the invariant holds throughout the algorithm.

Termination and output. Each loop iteration deletes exactly one vertex from the left side and one from the right side. Since $|A| = |B| = n$, after exactly n iterations the residual graph H is empty, so the loop terminates.

At termination, H is the empty bipartite graph. By (I4), the empty graph has a perfect matching with exactly t red edges. The only perfect matching of the empty graph is the empty set, whose red count is 0. Therefore $t = 0$. Using (I3), we conclude

$$r(M) = k.$$

Using (I2), every original vertex of G is the endpoint of exactly one edge of M . Hence M is a perfect matching of G . So when the algorithm outputs M , it is indeed a perfect matching with exactly k red edges.

If the algorithm outputs No in step 1, that is correct by definition of Dec .

Running time. There is one initial oracle call. During the while-loop, every tested edge is incident to the currently chosen left vertex a , and each left vertex is chosen at most once. Hence the total number of tested edges over the entire run is at most $m = |E|$. So the algorithm makes at most $m + 1$ calls to **Dec**.

Let

$$T^*(n, m) := \max\{T_{\text{Dec}}(n', m') : 0 \leq n' \leq n, 0 \leq m' \leq m\}.$$

Then the total running time is

$$O((m + 1)T^*(n, m) + m).$$

Therefore, if **Dec** is polynomial-time, then **Search** is polynomial-time as well.

Finally, Du's 2026 theorem supplies precisely such a deterministic polynomial-time decision algorithm for bipartite exact matching. Applying the above reduction to that decision routine yields the desired deterministic polynomial-time algorithm that both decides existence and outputs a witness matching when one exists. (arxiv.org) $\square$

Gap to the full statement

None. The algorithm above solves exactly the original bipartite search problem. The only caveat is methodological: the decision subroutine is cited rather than rederived here.

Self red-team

• Counterexample attempts.

- A single alternating 4-cycle with one all-blue perfect matching and one all-red perfect matching realizes red counts $\{0, 2\}$. The forcing lemma survives this test exactly: each edge belongs to an exact- t perfect matching if and only if the residual instance has target $t - \rho(e)$.
- Two disjoint copies realize $\{0, 2, 4\}$. This kills the seductive but false claim that achievable red counts must form an interval. My proof does not use any interval or convexity property of the red counts.
- I checked the degenerate cases $n = 0$, $n = 1$, all-blue, all-red, and disconnected graphs. The invariant proof handles all of them without modification.

• Three likely referee objections.

1. *“The hard part is imported from a recent preprint.”* Correct. I have flagged this explicitly. The independently proved part here is the witness-extraction reduction, together with the hypothesis audit showing that every residual subinstance lies within the scope of the cited theorem.
2. *“Why must the inner loop find a feasible edge?”* Because any exact witness perfect matching of the current residual graph matches the chosen vertex a to some neighbor b , and the forcing lemma shows that the corresponding residual oracle query must answer Yes.
3. *“Could choosing a feasible edge now make the instance infeasible later?”* No. The algorithm accepts an edge e only after the oracle certifies that the residual instance $H - \{a, b\}$ is itself a Yes-instance for the updated target.

• Boundary and scope checks.

- The proof correctly gives the empty matching when $n = 0$ and $k = 0$, and rejects when $n = 0$ and $k \neq 0$.
- If $k < 0$ or $k > n$, the decision routine rejects; the search wrapper does not introduce any new issue.
- The reduction itself is graph-agnostic, but the imported deterministic decision theorem is only for bipartite graphs. So the manuscript does *not* accidentally claim a deterministic polynomial-time algorithm for the general-graph exact matching problem.

Invoked results

- **Du (2026), Theorem 1.1 / Proposition 2.3.** Classification: SOLVES.

Statement used. The cited preprint states that Exact Matching on bipartite graphs admits a deterministic polynomial-time algorithm (the abstract claims $O(n^6)$); the general bipartite case is handled by tight-cut decomposition into brace blocks, blockwise determinant-interpolation tests, and a dynamic program over target partitions. (arxiv.org)

Citation. Yuefeng Du, *Bipartite Exact Matching in P*, arXiv:2604.01571, 2026.

Hypothesis-match audit. Every oracle instance in my proof is a bipartite graph obtained from the original bipartite graph by deleting one left vertex and one right vertex; hence it remains bipartite and still has equal side sizes. The red/blue edge coloring is inherited unchanged, and the updated target $t - \rho(e)$ is an integer. Therefore the theorem applies verbatim at every oracle call. I use only this decision consequence; I do not import the proof of the determinant nonvanishing theorem.

Self-confidence

3 — I checked the search reduction carefully and could not break it; my only real uncertainty comes from relying on a very recent external preprint for the decision subroutine rather than reproducing that proof here.